Method-related hooks
: method_added, method_missing, method_removed, method_undefined, singleton_method_added, singleton_method_removed, singleton_method_undefined

Class and module-related hooks
: append_features, const_missing, extend_object, extended, included, inherited, initialize_clone, initialize_copy, initialize_dup

Object marshaling hooks
: marshal_dump, marshal_load

Coercion hooks
: coerce, induced_from, to_xxx

We won't discuss all of them in this chapter—instead, we'll show just a few examples of use.

The inherited Hook

If a class defines a class method called inherited, Ruby will call that method whenever that class is subclassed (that is, whenever any class inherits from the original).

This hook is often used in situations where a base class needs to keep track of its children. For example, an online store might offer a variety of shipping options. Each might be represented by a separate class, and each of these classes could be a subclass of a single Shipping class. This parent class could keep track of all the various shipping options by recording every class that subclasses it. When it comes time to display the shipping options to the user, the application could call the base class, asking it for a list of its children:

metaprogramming/shipping.rb

```
class Shipping # Base class
  @children = [] # this variable is in the class, not instances

  def self.inherited(child)
    @children << child
  end

  def self.shipping_options(weight, international)
    @children.select { |child| child.can_ship(weight, international) }
  end
end

class MediaMail < Shipping
  def self.can_ship(_weight, international)
    !international
  end
end

class FlatRatePriorityEnvelope < Shipping
  def self.can_ship(weight, international)
    weight < 64 && !international
  end
end

class InternationalFlatRateBox < Shipping
  def self.can_ship(weight, international)
    weight < 9*16 && international
  end
end
```